

INSTITUTO INFNET
ESCOLA SUPERIOR DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE SOFTWARE



Projeto de Bloco: Sistemas Digitais Embarcados

[26E2_2]

TP1

ALUNO: João Paulo Fonseca de Araújo

E-MAIL: joao.paraujo@al.infnet.edu.br

1. Título do Projeto e Visão Geral

Título: Controlador VGA com Framebuffer ARM→FPGA

O projeto consiste em um sistema de exibição de vídeo onde o processador ARM e o FPGA cooperam de forma complementar. O **Raspberry Pi Zero 2W** (ARM Cortex-A53, AArch64, Linux 64-bit) atua como CPU host, responsável por gerar o conteúdo a ser exibido — seja padrões geométricos, texto ou animações. O **Tang Nano 4K** (FPGA Gowin GW1NSR-4C) assume o papel de controlador de hardware: gera os sinais de sincronismo VGA (hsync e vsync), mantém o framebuffer em BRAM e realiza a varredura de pixels para o monitor.

A comunicação entre os dois dispositivos ocorre via barramento **SPI** (GPIO do RPi → pinos do FPGA), com o ARM enviando dados de pixel diretamente para a memória BRAM do FPGA. O sistema produz saída de vídeo estável na resolução **640×480 pixels @ 60 Hz** via conector VGA, requerendo apenas um cabo VGA padrão e um monitor compatível.

2. Objetivos Funcionais e Não Funcionais

2.1 Objetivos Funcionais

OF-01	Gerar sinais VGA 640×480 @ 60 Hz estáveis no FPGA (hsync, vsync, RGB)
OF-02	Manter framebuffer em BRAM do FPGA com dados escritos pelo ARM via SPI
OF-03	Exibir padrões estáticos (barras de cor, xadrez) gerados pelo FPGA de forma autônoma
OF-04	Exibir imagens geradas pelo ARM: padrões, gradientes, formas geométricas
OF-05	Renderizar texto (fonte bitmap 8×8) no framebuffer via rotina ARM
OF-06	Produzir animação simples: objeto em movimento ou transição de cor

2.2 Objetivos Não Funcionais

ONF-01	Estabilidade de sincronismo: sem tearing ou flickering perceptível no monitor
ONF-02	Latência ARM→monitor menor que 2 frames (< 33 ms para 60 Hz)
ONF-03	Uso de BRAM Gowin dentro dos limites do GW1NSR-4C (180 Kb disponíveis)

ONF-04	Código Verilog sintetizável e documentado; Assembly ARM com comentários funcionais
--------	--

3. Arquitetura Preliminar do Sistema

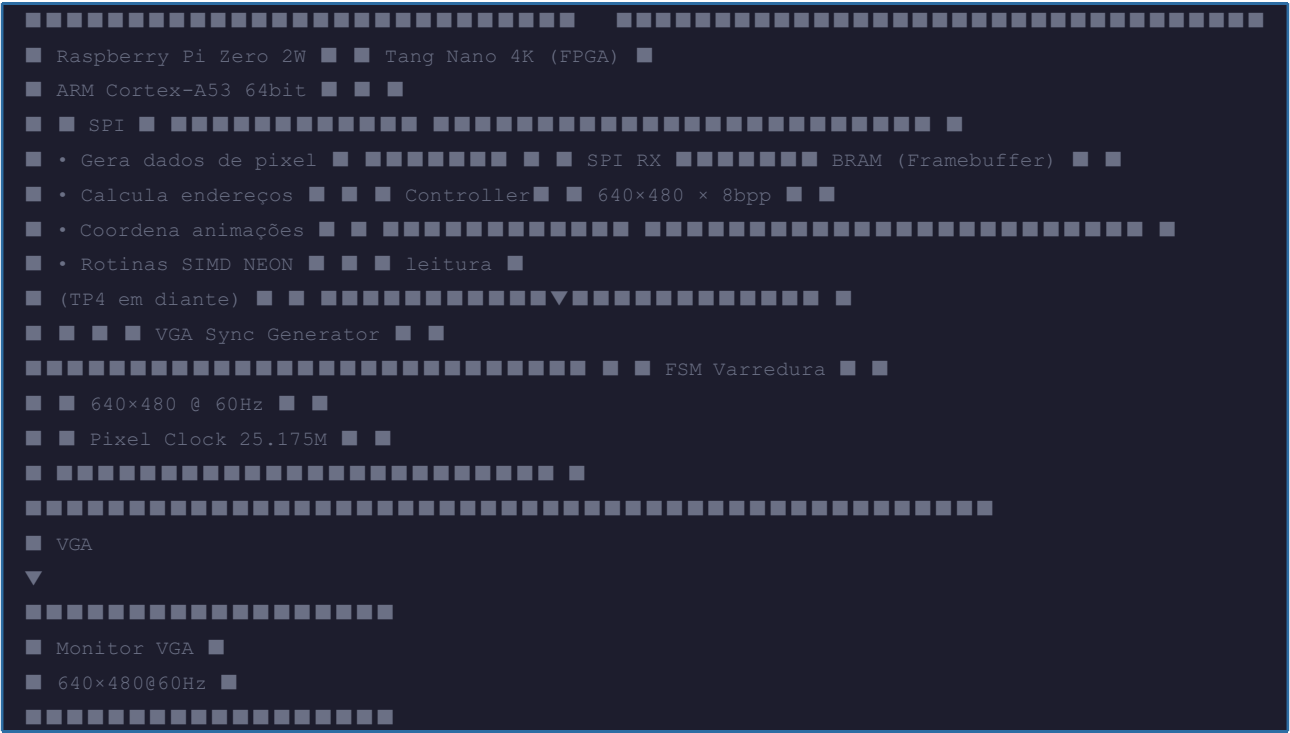
3.1 Visão Geral dos Componentes

O sistema é composto por dois subsistemas principais interligados via SPI:

Componente	Função	Justificativa
RPi Zero 2W (ARM Cortex-A53)	Host CPU: gera padrões de pixel, coordena frames, executa lógica de animação e envia dados via SPI	Disponível no kit; suporte a Linux 64-bit; fácil programação em Assembly AArch64 e C
Tang Nano 4K (Gowin GW1NSR-4C)	FPGA: recebe pixels via SPI, armazena em BRAM, gera sinais VGA e realiza varredura de frame	BRAM suficiente para framebuffer 640×480×8bpp; PLL interno para pixel clock 25.175 MHz; conector VGA nativo no kit
BRAM interna	Framebuffer: armazena o conteúdo atual do frame a ser exibido	Caso de uso canônico de BRAM; acesso dual-port permite escrita (SPI) e leitura (VGA) simultâneas sem arbitragem complexa
SPI Bus (GPIO)	Canal de comunicação ARM→FPGA: transferência de dados de pixel e comandos de controle	Simples de implementar em ambos os lados; Linux spidev disponível no RPi; frequência ajustável (1–10 MHz)

3.2 Diagrama de Fluxo de Dados

O fluxo de dados percorre o seguinte caminho, do ARM ao monitor:



3.3 Timing VGA 640×480 @ 60 Hz

A resolução 640×480 @ 60 Hz requer pixel clock de **25.175 MHz**. Os parâmetros de timing são bem estabelecidos e amplamente suportados por monitores:

Parâmetro	Horizontal (pixels)	Vertical (linhas)
Região visível	640	480
Front porch	16	10
Sync pulse	96	2
Back porch	48	33
Total	800	525

4. Configuração do Ambiente de Desenvolvimento

4.1 Raspberry Pi Zero 2W — ARM Host

O Raspberry Pi OS 64-bit (Bookworm, Debian 12) é instalado via Raspberry Pi Imager. Após o boot, as ferramentas ARM são instaladas e o acesso SSH configurado:

```
# Atualizar sistema
sudo apt update && sudo apt upgrade -y

# Instalar ferramentas Assembly ARM 64-bit
sudo apt install -y gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu
sudo apt install -y gdb-multiarch build-essential

# Verificar toolchain
aarch64-linux-gnu-as --version # GNU Assembler
aarch64-linux-gnu-gcc --version # GCC cross-compiler
aarch64-linux-gnu-gdb --version # GDB debugger

# Compilar Assembly ARM nativo (no proprio RPi Zero 2W)
gcc -o framebuffer_calc assembly_tpl/framebuffer_calc.s -no-pie
./framebuffer_calc
```

4.2 Tang Nano 4K — FPGA

A Gowin IDE (versão 1.9.x para Linux) é instalada para síntese, place-and-route e programação da Tang Nano 4K. O simulador Icarus Verilog é utilizado para testes rápidos sem necessidade de síntese:

```
# Instalar Icarus Verilog (simulacao rapida)
sudo apt install -y iverilog gtkwave

# Simular modulo color_selector
cd verilog_tpl
iverilog -o sim color_selector.v tb_color_selector.v
vvp sim

# Saida esperada: todos os testes PASS

# Visualizar formas de onda
gtkwave tb_color_selector.vcd &

# Gowin IDE: sintetizar e gravar na Tang Nano 4K
# 1. Criar projeto Gowin, selecionar GW1NSR-4C
# 2. Adicionar color_selector.v
# 3. Run Synthesis → Place & Route → Generate Bitstream
# 4. Programmer → selecionar porta USB → Download
```

4.3 Repositório Git

```
# Estrutura do repositorio
vga-framebuffer-arm-fpga/
  ■■■ assembly_tpl/
  ■■■■ framebuffer_calc.s # Programa ARM64 Assembly
  ■■■ verilog_tpl/
  ■■■■ color_selector.v # Modulo combinacional
  ■■■■ tb_color_selector.v # Testbench
```

5. Módulo Verilog: color_selector

```
■■■ docs/ # Relatorios e diagramas
■■■ screenshots/ # Evidencias de funcionamento
■■■ README.md

# Commits semanticos
git commit -m 'feat(verilog): adiciona modulo color_selector com testbench'
git commit -m 'feat(asm): adiciona calculadora de enderecos framebuffer ARM64'
git commit -m 'docs: adiciona relatorio TP1 e planejamento'
```

5.1 Descrição e Propósito

O módulo **color_selector** é um circuito puramente combinacional que implementa um gerador de barras de cor verticais, dividindo a resolução horizontal de 640 pixels em 8 regiões de 80 pixels cada, com uma cor distinta por região. Este módulo representa o núcleo da lógica de geração de padrões que evoluirá para o gerador VGA completo no TP2.

A escolha de um módulo combinacional puro (sem clock) é pedagogicamente correta para TP1: permite validar a lógica de seleção de cor via simulação, verificar a cobertura de todas as 8 regiões e garantir que as saídas RGB respondam corretamente a qualquer entrada de pixel.

5.2 Interface do Módulo

Sinal	Dir.	Largura	Descrição
pixel_x	IN	10 bits	Coordenada horizontal do pixel atual (0..639)
pixel_y	IN	9 bits	Coordenada vertical do pixel atual (0..479)
red	OUT	4 bits	Canal vermelho do pixel (0..15 → 0..255 após expansão)
green	OUT	4 bits	Canal verde do pixel
blue	OUT	4 bits	Canal azul do pixel

5.3 Snippet Principal do Módulo

```
module color_selector (
    input wire [9:0] pixel_x, // coordenada X: 0..639
    input wire [8:0] pixel_y, // coordenada Y: 0..479
    output reg [3:0] red,
    output reg [3:0] green,
    output reg [3:0] blue
);
    wire [3:0] region;
    assign region = pixel_x[9:3] / 10; // 8 regioes de ~80px

    always @(*) begin // logica combinacional pura
        case (region)
            4'd0: begin red=4'hF; green=4'hF; blue=4'hF; end // Branco
```

```

4'd1: begin red=4'hF; green=4'hF; blue=4'h0; end // Amarelo
4'd2: begin red=4'h0; green=4'hF; blue=4'hF; end // Ciano
4'd3: begin red=4'h0; green=4'hF; blue=4'h0; end // Verde
4'd4: begin red=4'hF; green=4'h0; blue=4'hF; end // Magenta
4'd5: begin red=4'hF; green=4'h0; blue=4'h0; end // Vermelho
4'd6: begin red=4'h0; green=4'h0; blue=4'hF; end // Azul
default: begin red=4'h0; green=4'h0; blue=4'h0; end // Preto
endcase
end
endmodule

```

5.4 Resultado da Simulação (Testbench)

O testbench **tb_color_selector** verifica 18 casos de teste, cobrindo todas as 8 barras de cor e a independência da coordenada Y. A saída esperada no terminal é:

```

=====
Testbench: color_selector - VGA Color Bars
Resolucao: 640x480 | 8 barras de 80px
=====

[PASS] Barra BRANCO x= 0 -> R=f G=f B=f
[PASS] Barra BRANCO x= 40 -> R=f G=f B=f
[PASS] Barra AMARELO x= 80 -> R=f G=f B=0
[PASS] Barra CIANO x=160 -> R=0 G=f B=f
[PASS] Barra VERDE x=240 -> R=0 G=f B=0
[PASS] Barra MAGENTA x=320 -> R=f G=0 B=f
[PASS] Barra VERMELHO x=400 -> R=f G=0 B=0
[PASS] Barra AZUL x=480 -> R=0 G=0 B=f
[PASS] Barra PRETO x=560 -> R=0 G=0 B=0
...
=====
Resultado: 18 PASS | 0 FAIL
STATUS: TODOS OS TESTES PASSARAM
=====

```

6. Programa Assembly ARM 64-bit: framebuffer_calc

6.1 Descrição e Propósito

O programa **framebuffer_calc.s** é escrito em Assembly AArch64 puro e executado nativamente no Raspberry Pi Zero 2W. Ele demonstra operações fundamentais de cálculo de endereços de pixel em um framebuffer linear, que é a base de toda a comunicação ARM→FPGA nos TPs subsequentes.

A fórmula central é: **addr = y × FB_WIDTH + x**, onde FB_WIDTH = 640 e o resultado representa o deslocamento em bytes no framebuffer de 307.200 bytes (640×480×1 byte). O programa calcula e exibe endereços decimais e hexadecimais para 8 pixels de teste.

6.2 Conceitos ARM64 Demonstrados

Conceito	Instrução / Mecanismo	Uso no programa
Multiplicação	MUL x24, x22, x23	Calcula $y \times \text{FB_WIDTH}$
Adição	ADD x24, x24, x21	Soma o offset X ao resultado
Load reg/imediato	MOV x23, #640 / LDR x1, =label	Carrega constante e endereço de string
Loop com contador	SUB x20, x20, #1 / CBZ x20, done	Itera 8 vezes sobre tabela de pixels
Tabela de dados	.hword val (ldrh [x19], #2)	Acessa tabela de pares (x,y) de 16 bits
Syscall write()	MOV x8, #64 / SVC #0	Saída de texto via Linux AArch64 syscall
Convenção de chamada	STP/LDP x29, x30 [sp]	Salva/restaura frame pointer e link register

6.3 Snippet Principal — Loop de Cálculo de Endereço

```
loop_pixels:
cbz x20, loop_done // termina se contador == 0

ldrh w21, [x19], #2 // carrega x (16 bits) da tabela
ldrh w22, [x19], #2 // carrega y (16 bits) da tabela

// Calcula: addr = y * FB_WIDTH + x
mov x23, #640 // FB_WIDTH = 640
mul x24, x22, x23 // x24 = y * 640
add x24, x24, x21 // x24 = y * 640 + x (endereço final)

// Imprime resultado formatado
bl print_uint // endereço decimal
bl print_hex // endereço hexadecimal

sub x20, x20, #1 // decrementa contador
b loop_pixels
```

6.4 Saída do Programa no Raspberry Pi Zero 2W

```
=====
VGA Framebuffer TP1 - ARM64 Assembly
Resolucao: 640x480 | Modo: 8bpp
=====

[INFO] Tamanho total do framebuffer: 307200 bytes

Pixel (x, y) | Endereco (dec) | Offset hex
-----+-----+-----
Pixel (0, 0) | addr = 0 | 0x0
Pixel (320, 240) | addr = 154240 | 0x25a80
Pixel (639, 479) | addr = 307199 | 0x4afff
Pixel (10, 5) | addr = 3210 | 0xc8a
Pixel (100, 100) | addr = 64100 | 0xfa64
Pixel (200, 300) | addr = 192200 | 0x2ee48
Pixel (0, 479) | addr = 306560 | 0x4ad80
Pixel (639, 0) | addr = 639 | 0x27f

[OK] Formula: addr = y * FB_WIDTH + x
```


[OK] Programa encerrado com sucesso.